

Entorno personal de aprendizaje:

CREACIÓN DE PÁGINAS WEB



Información SEDE 1



Ministerio de
**TECNOLOGÍAS
DE LA INFORMACIÓN
Y COMUNICACIÓN**

Dirección: Complejo Santos E2 - Gral. Santos 1170 c/ Concordia
Teléfono: (+595 21) 217 9000
Correo electrónico: comunicacion@mitic.gov.py Mesa de entrada: sgeneral@mitic.gov.py
Asunción – Paraguay

Curso de creación de páginas web: Unidad 6 Fundamentos de JavaScript.

Todas las imágenes presentes en este material de apoyo educativo han sido reconocidas y, en todos los casos, son de uso libre. Todas las fuentes bibliográficas utilizadas en este material de apoyo educativo han son citadas mediante el gestor de referencias bibliográficas Zotero, desarrollado por el Center for History and New Media de la Universidad George Mason, utilizando el estilo de la American Psychological Association Séptima Edición.



Ministerio de
**TECNOLOGÍAS
DE LA INFORMACIÓN
Y COMUNICACIÓN**

 **GOBIERNO
NACIONAL**

*Paraguay
de la gente*

Unidad 6 Javascript y Low/No Code

Contenido

1. Programación JavaScript: Ejecución y Sintaxis.....	4
2. Sintaxis Avanzada de JavaScript	5
3. Estructuras de datos en JavaScript.....	7
4. Funciones y estructuras de control de JavaScript	9
4.1. Funciones en JavaScript	10
4.2. Estructuras de Control en JavaScript	10
4.3. Bucles (for, while, do-while).....	11
4.4. Switch	11
5. Modificación de JavaScript del contenido y estilo del sitio web.....	12
5.1. Cambiar el Contenido de un Elemento	12
5.2. Crear y Agregar Nuevos Elementos.....	12
5.3. Eliminar Elementos	13
6. Manipulación del Estilo CSS	13
6.1. Cambiar Propiedades de Estilo	13
6.2. Agregar o Quitar Clases CSS	13
6.3. Eventos del Usuario y Manipulación Dinámica	14
Referencias bibliográficas	15

Unidad 6 Javascript y Low/No Code

1. Programación JavaScript: Ejecución y Sintaxis

JavaScript es un lenguaje flexible que permite desarrollar desde simples interacciones hasta complejas aplicaciones web. Con estas estructuras básicas y avanzadas, puedes comenzar a explorar sus posibilidades y mejorar tus habilidades en programación web.

Aquí tienes una descripción de la sintaxis básica junto con ejemplos:

- **Comentarios:**

Descripción: Los comentarios en JavaScript son texto que el intérprete ignora al ejecutar el código. Se utilizan para hacer anotaciones o explicaciones dentro del código.

Ejemplo:

```
// Esto es un comentario de una sola línea
```

```
/* Esto es un comentario de múltiples líneas */
```

- **Variables:**

Descripción: Las variables se utilizan para almacenar datos. En JavaScript, no es necesario declarar el tipo de variable.

Ejemplo:

```
var nombre = "Juan";  
let edad = 30;  
const PI = 3.1416; // Constante
```

- **Tipos de Datos:**

Descripción: JavaScript admite diferentes tipos de datos como números, cadenas de texto, booleanos, arrays, objetos, entre otros.

Ejemplo:

Unidad 6 Javascript y Low/No Code

```
var numero = 10;
var texto = "Hola";
var esActivo = true;
var colores = ["rojo", "verde", "azul"];
var persona = {
  nombre: "Ana",
  edad: 25
};
```

- **Operadores:**

Descripción: JavaScript tiene operadores aritméticos (+, -, *, /), de asignación (=), de comparación (==, ===, !=, !==, >, <, >=, <=), lógicos (&&, ||, !), entre otros.

Ejemplo:

```
var resultado = 10 + 5; // Operador aritmético
var esMayor = (edad > 18); // Operador de comparación
var esVerdad = (true && false); // Operador lógico
```

2. Sintaxis Avanzada de JavaScript

JavaScript también ofrece características más avanzadas que permiten un código más eficiente y expresivo:

- **Funciones:**

Descripción: Las funciones en JavaScript son bloques de código reutilizables que se pueden ejecutar cuando se llaman. Pueden aceptar parámetros y devolver valores.

Ejemplo:

```
function saludar(nombre) {
  return "¡Hola, " + nombre + "!";
}
var mensaje = saludar("María"); // Devuelve "¡Hola, María!"
```

- **Expresiones Condicionales:**

Descripción: Permiten ejecutar diferentes bloques de código según una

Unidad 6 Javascript y Low/No Code

condición booleana.

Ejemplo:

```
var edad = 20;
if (edad >= 18) {
  console.log("Eres mayor de edad");
} else {
  console.log("Eres menor de edad");
}
```

- **Bucles:**

Descripción: Permiten ejecutar un bloque de código varias veces mientras se cumpla una condición especificada.

Ejemplo:

```
for (var i = 0; i < 5; i++) {
  console.log("Número: " + i);
}
```

- **Manejo de Errores (Try-Catch):**

Descripción: Permite manejar errores que pueden surgir durante la ejecución del código.

Ejemplo:

```
try {
  // Código que puede generar un error
  var resultado = 10 / 0; // Error: división por cero
} catch (error) {
  console.log("Error: " + error.message);
}
```

- **Programación Orientada a Objetos:**

Descripción: JavaScript soporta programación orientada a objetos mediante la creación de clases y la definición de métodos y propiedades.

Ejemplo:

Unidad 6 Javascript y Low/No Code

```
class Persona {
  constructor(nombre, edad) {
    this.nombre = nombre;
    this.edad = edad;
  }
  saludar() {
    return "¡Hola, soy " + this.nombre + "!";
  }
}
var personal = new Persona("Carlos", 35);
console.log(personal.saludar()); // Devuelve "¡Hola, soy Carlos!"
```

3. Estructuras de datos en JavaScript

En JavaScript, las estructuras de datos son fundamentales para almacenar y organizar información de manera eficiente. A continuación, te presento algunas de las estructuras de datos más comunes en JavaScript:

- **Arrays**

Los arrays son listas ordenadas de elementos que permiten almacenar múltiples valores en una sola variable. Los elementos pueden ser de cualquier tipo de datos y se accede a ellos mediante un índice numérico.

Ejemplo:

```
var frutas = ["manzana", "banana", "naranja"];
var numeros = [1, 2, 3, 4, 5];
var mixto = ["Hola", 123, true];
console.log(frutas[0]); // Accede al primer elemento:
"manzana"
console.log(numeros.length); // Devuelve la longitud del
array: 5
}
var personal = new Persona("Carlos", 35);
console.log(personal.saludar()); // Devuelve "¡Hola, soy Carlos!"
```

Unidad 6 Javascript y Low/No Code

- **Objetos**

Los objetos en JavaScript son estructuras de datos que permiten almacenar datos y métodos relacionados. Se definen utilizando pares clave-valor y pueden contener propiedades y métodos.

Ejemplo:

```
var persona = {  
  nombre: "Juan",  
  edad: 30,  
  profesion: "Ingeniero",  
  saludar: function() {  
    return "Hola, soy " + this.nombre;  
  }  
};  
console.log(persona.nombre); // Accede a la propiedad  
"nombre": "Juan"  
console.log(persona.saludar()); // Llama al método saludar():  
"Hola, soy Juan"
```

- **Sets**

Los sets son colecciones de valores únicos, lo que significa que no pueden contener elementos duplicados. Son útiles para almacenar listas de valores únicos y proporcionan métodos para añadir, eliminar y verificar la existencia de elementos.

Ejemplo:

```
var numerosUnicos = new Set();  
numerosUnicos.add(1);  
numerosUnicos.add(2);  
numerosUnicos.add(3);  
numerosUnicos.add(1); // No se añade porque ya existe  
  
console.log(numerosUnicos.size); // Devuelve el tamaño del  
set: 3
```

- **Maps**

Unidad 6 Javascript y Low/No Code

Los maps son colecciones de pares clave-valor donde tanto la clave como el valor pueden ser de cualquier tipo de datos. A diferencia de los objetos, los maps conservan el orden de inserción de los elementos.

Ejemplo:

```
var mapa = new Map();
mapa.set("a", 1);
mapa.set("b", 2);
mapa.set("c", 3);

console.log(mapa.get("b")); // Devuelve el valor asociado a
la clave "b": 2
console.log(mapa.size); // Devuelve el tamaño del mapa: 3
```

- **Arrays Asociativos (Objetos como Maps)**

En JavaScript, los objetos pueden utilizarse como maps o arrays asociativos, donde las claves son cadenas de texto y los valores pueden ser cualquier tipo de datos. Esta estructura es comúnmente utilizada para almacenar y acceder a datos de manera dinámica.

Ejemplo:

```
var persona = {
  nombre: "Ana",
  edad: 25,
  profesion: "Diseñadora"
};

console.log(persona["nombre"]); // Accede a la propiedad
usando corchetes: "Ana"
console.log(persona.edad); // Accede a la propiedad
directamente: 25
```

4. Funciones y estructuras de control de JavaScript

En JavaScript, las funciones y las estructuras de control son fundamentales para controlar el flujo de ejecución del código y para encapsular lógica reutilizable. A continuación, te presento una descripción y ejemplos de funciones y estructuras de control en JavaScript:

4.1.Funciones en JavaScript

Las funciones en JavaScript son bloques de código que se pueden ejecutar cuando se las llama. Pueden aceptar parámetros y devolver valores, lo que las hace muy útiles para encapsular y reutilizar código.

- **Declaración de Funciones**

Las funciones se pueden declarar de varias formas en JavaScript:

```
function saludar(nombre) {  
    return "Hola, " + nombre + "!";  
}  
  
var mensaje = saludar("Juan");  
console.log(mensaje); // Devuelve "Hola, Juan!"
```

- **Expresión de Función:**

```
var sumar = function(a, b) {  
    return a + b;  
};  
  
var resultado = sumar(3, 5);  
console.log(resultado); // Devuelve 8
```

- **Funciones Flecha (Arrow Functions):**

```
var multiplicar = (x, y) => x * y;  
var producto = multiplicar(4, 6);  
console.log(producto); // Devuelve 24
```

4.2.Estructuras de Control en JavaScript

Las estructuras de control permiten tomar decisiones y repetir bloques de código según condiciones específicas.

- **Condicionales (if, else if, else)**

Las estructuras condicionales se utilizan para ejecutar diferentes bloques de código según el cumplimiento de una condición booleana.

```
var edad = 20;
```

Unidad 6 Javascript y Low/No Code

```
if (edad >= 18) {  
  console.log("Eres mayor de edad");  
} else {  
  console.log("Eres menor de edad");  
}
```

4.3.Bucles (for, while, do-while)

Los bucles permiten ejecutar un bloque de código varias veces hasta que se cumpla una condición específica.

- **Bucle for:**

```
for (var i = 0; i < 5; i++) {  
  console.log("Iteración " + i);  
}
```

- **Bucle while:**

```
var contador = 0;  
  
while (contador < 3) {  
  console.log("Contador: " + contador);  
  contador++;  
}
```

- **Bucle do-while:**

```
var num = 1;  
  
do {  
  console.log("Número: " + num);  
  num++;  
} while (num <= 5);
```

4.4.Switch

La sentencia switch se utiliza para realizar diferentes acciones basadas en múltiples condiciones posibles de una sola variable.

```
var diaSemana = 2;  
var nombreDia;  
  
switch (diaSemana) {  
  case 1:  
    nombreDia = "Lunes";  
    break;  
  case 2:
```

Unidad 6 Javascript y Low/No Code

```
    nombreDia = "Martes";  
    break;  
case 3:  
    nombreDia = "Miércoles";  
    break;  
default:  
    nombreDia = "Día no válido";  
}  
console.log(nombreDia); // Devuelve "Martes"
```

5. Modificación de JavaScript del contenido y estilo del sitio web

JavaScript permite modificar el contenido de los elementos HTML, ya sea cambiando el texto, agregando nuevos elementos o eliminando existentes.

5.1. Cambiar el Contenido de un Elemento

Puedes cambiar el contenido de un elemento HTML usando propiedades como `innerHTML`, `textContent` o manipulando directamente propiedades específicas del elemento.

- Ejemplo con `innerHTML`:

```
// Cambiar el contenido de un elemento con id "titulo"  
document.getElementById("titulo").innerHTML = "Nuevo Título";
```

- Ejemplo con `textContent`:

```
// Cambiar el texto de un elemento con clase "mensaje"  
document.querySelector(".mensaje").textContent = "Mensaje actualizado";
```

5.2. Crear y Agregar Nuevos Elementos

Puedes crear nuevos elementos HTML dinámicamente usando JavaScript y luego agregarlos al documento.

- Ejemplo de creación y adición de un nuevo párrafo:

```
// Crear un nuevo elemento <p>  
var nuevoParrafo = document.createElement("p");  
// Agregar texto al párrafo  
nuevoParrafo.textContent = "Este es un nuevo párrafo";
```

Unidad 6 Javascript y Low/No Code

```
// Agregar el nuevo párrafo al final del cuerpo del documento
document.body.appendChild(nuevoParrafo);
```

5.3.Eliminar Elementos

También puedes eliminar elementos existentes del documento HTML usando métodos como `remove()`.

Ejemplo de eliminación de un elemento por su id:

```
// Eliminar un elemento con id "parrafoEliminar"
var elementoEliminar =
document.getElementById("parrafoEliminar");
elementoEliminar.remove();
```

6. Manipulación del Estilo CSS

JavaScript permite cambiar y manipular los estilos CSS de los elementos HTML dinámicamente. Puedes modificar propiedades individuales o aplicar y quitar clases CSS.

6.1.Cambiar Propiedades de Estilo

Puedes cambiar el estilo de un elemento directamente modificando sus propiedades CSS a través de JavaScript.

```
// Cambiar el color de fondo de un elemento con id "caja"
document.getElementById("caja").style.backgroundColor =
"blue";
```

6.2.Agregar o Quitar Clases CSS

Manipular las clases CSS de un elemento permite aplicar estilos predefinidos o modificar varios estilos a la vez.

- Ejemplo de agregar una clase:

```
// Agregar la clase "destacado" a un elemento con id
"elemento"
document.getElementById("elemento").classList.add("destacado")
);
```

- Ejemplo de quitar una clase:

Unidad 6 Javascript y Low/No Code

```
// Quitar la clase "oculto" de un elemento con id  
"otroElemento"  
document.getElementById("otroElemento").classList.remove("ocu  
lto");
```

6.3.Eventos del Usuario y Manipulación Dinámica

JavaScript también permite manejar eventos del usuario, como clics o cambios en los elementos, para realizar manipulaciones dinámicas basadas en la interacción del usuario.

- Ejemplo de manejar clics en un botón:

```
// Agregar un evento de clic a un botón con id "boton"  
document.getElementById("boton").addEventListener("click",  
function() {  
    alert("¡Haz clic en el botón!");  
});
```

Unidad 6 Javascript y Low/No Code

Referencias bibliográficas

Equipo editorial de IONOS. (2022, 27 septiembre). *Resumen de los comandos Java más importantes*. IONOS Digital Guide. <https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/comandos-java/>

Command en Java / Patrones de diseño. (s. f.). <https://refactoring.guru/es/design-patterns/command/java/example>

Plynko, P. (2024, 1 febrero). *Argumentos de la línea de comandos de Java*. CodeGym. <https://codegym.cc/es/groups/posts/es.664.argumentos-de-la-linea-de-comandos-de-java>

Java: cómo compilar y ejecutar desde línea de comandos | DAM. (s. f.). <https://ies-pablo-serrano.gitbook.io/dam/java-como-compilar-y-ejecutar-desde-linea-de-comandos>